CHAPTER 1

An Anatomy of a Large-scale Image Search Engine

Wei-Cheng Lai, Edward Chang, and Kwang-Ting (Tim) Cheng
VIMA Technologies.
3944 State Street, Suite #340
Santa Barbara, CA 93105
{*wlai,echang,timcheng*}*@vimatech.com*

The content of the World-Wide-Web has moved rapidly from text-only to multimedia-rich. As information becomes available more and more as multimedia content, and requires more personalized access, we deem the existing infrastructures inadequate. For enabling effective personalized searches in Web-based or large-scale image libraries, this chapter proposes a *perception-based search paradigm*. A search engine built on such a paradigm can *learn* users' subjective query concepts quickly through an intelligent sampling process. We present an anatomy of a large-scale image search engine, which includes a perception-based search engine, a multi-resolution image-feature extractor, and a high-dimensional indexer, as well as traditional components such as a *crawler* and *keyword extractor*. We depict a prototype that we have built and demonstrate its superiority over traditional systems in three respects: personalization, search accuracy, and efficiency.
Keywords:active learning, perception-based image retrieval, relevance feedback.

1. Introduction

The proliferation of digital cameras and the deployment of broadband infrastructure have drastically increased the multimedia content on the Web. Searching multimedia information, however, can be much more challenging than searching text documents. This is partly because multimedia content can be difficult to articulate, and partly because articulation can be subjective. For instance, it is difficult to describe a desired image using low-level

features such as color, shape, texture, and keywords. Moreover, different users may perceive the same image differently; and even if an image is perceived similarly, users may use different vocabulary (i.e., different combinations of low-level features) to depict it. It is thus necessary to build intelligent search engines that can provide the following two capabilities: *alleviating the need for users to specify complex query concepts*, and *supporting personalized searches.*

For enabling user-friendly query-concept formulation and effective personalized searches in Web-based or large-scale image libraries, this chapter presents a *perception-based search paradigm.* A search engine built on such a paradigm can *learn* users' subjective query concepts quickly through an intelligent sampling process. We first present two active learning algorithms, MEGA and SVM_{Active}, that we recently developed. We then propose a novel scheme that combines these two algorithms to learn a query concept in a small number of user iterations. To make both concept-learning and image retrieval efficient, we employ a multi-resolution image-feature extractor (Section 2.1), and a high-dimensional indexer (Section 2.2). Together with some traditional search-engine components— including crawler, keyword extractor, and user interface— we present an anatomy of a large-scale image search engine. Through examples and empirical study, we show that our system is superior over traditional image search engines in three aspects: personalization, search accuracy, and efficiency.

1.1. *An Illustrative Example*

In this example, we compare a keyword-based image retrieval system with our proposed perception-based image retrieval system. We use the Yahoo! Picture Gallery (i.e., http://gallery.yahoo.com) as a test site for keyword-based image retrieval. Suppose a user wants to retrieve images related to "bird of paradise." Given the keywords "bird of paradise" at the test site, the gallery engine retrieves five images of this flower as shown in Figure 1.

However, there are more than five images relevant to "bird of paradise" in the Yahoo image database. We will show shortly that our proposed system can retrieve more of these relevant images by taking the following steps: First, we query Yahoo's keyword-based search engine using "bird" and store the returned images (both birds and flowers) in a local database. Second, we apply our perception-based search engine to the local database. The learning steps for grasping the concept "bird of paradise" involves four

screens that are illustrated in the following four figures.

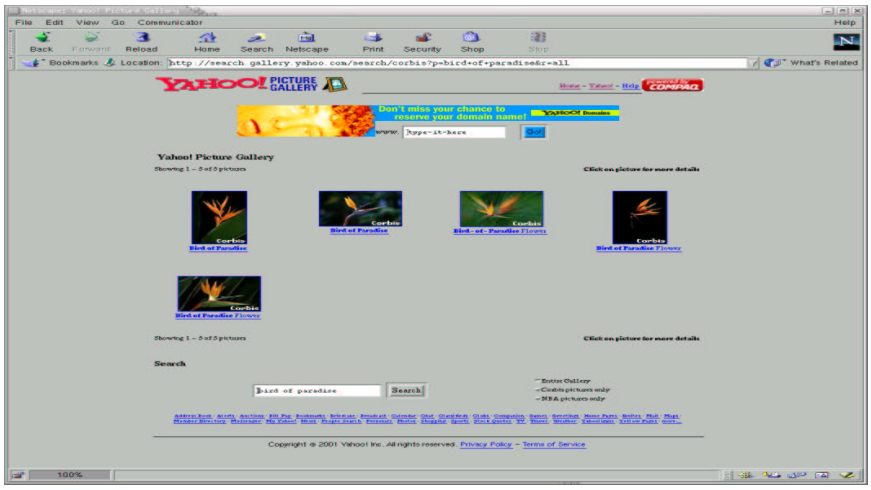


Fig. 1. "Bird of Paradise" Query Using Keyword-based Search Engine.



Fig. 2. "Bird of Paradise" Query Screen #1.

- Screen 1. Initial Screen. The system presents the initial screen to the user as depicted in Figure 2. The screen is split into two frames vertically. On the left-hand side of the screen is the learner frame;

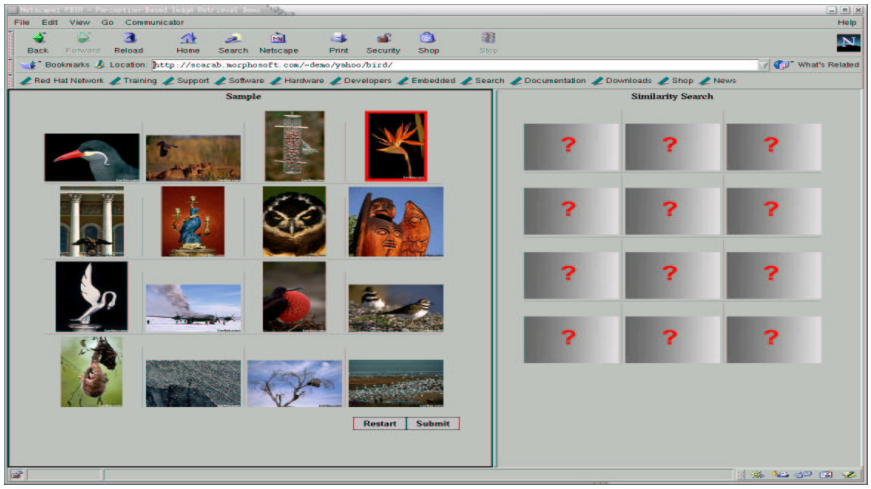


Fig. 3. "Bird of Paradise" Query Screen #2.

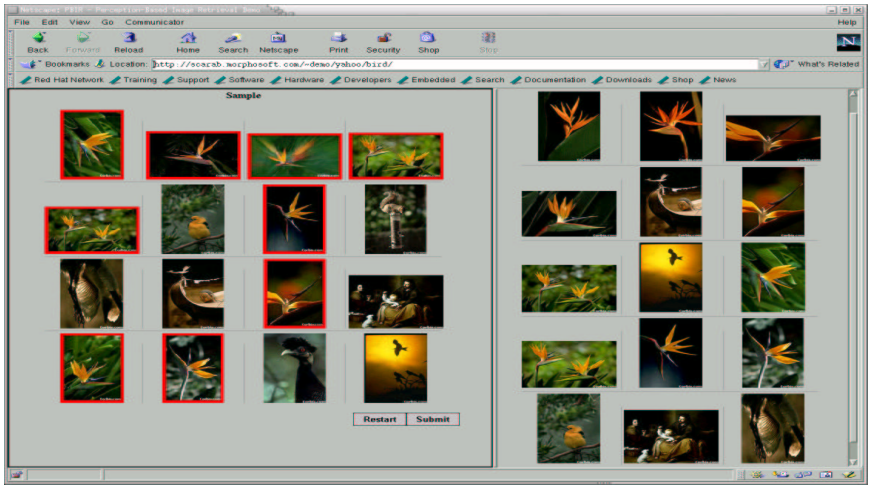


Fig. 4. "Bird of Paradise" Query Screen #3.

on the right-hand side is the similarity search frame. Through the learner frame, the system learns what the user wants via an active learning process. The similarity search frame displays images that match the user's query concept.

- Screen 2. Sampling and relevance feedback starts. Once the user clicks the "submit" button in the initial frame, the active learning

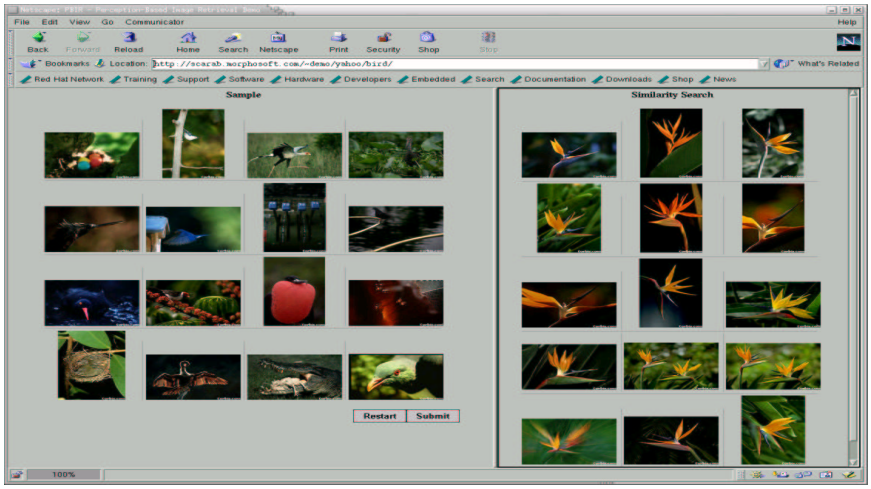


Fig. 5. "Bird of Paradise" Query Screen #4.

step commences to learn what the user wants. The system presents a number of samples in the learner frame, and the user marks images that are relevant to his/her query concept by clicking on the relevant images. As shown in Figure 3, one image (the last image in the first row) is selected as relevant, and the rest of the unmarked images are considered irrelevant. The user indicates the end of his/her selection by clicking on the submit button in the learner screen. This action brings up the next screen.

- Screen 3. Sampling and relevance feedback continues. Figure 4 shows the third screen. First, the similarity search frame displays what the system thinks will match the user's query concept at this time. As the figure indicates, eleven returned images fit the concept of "bird of paradise." The user's query concept has been captured, though somewhat fuzzily. The user can ask the system to further refine the target concept by selecting relevant images in the learner frame. In this example, nine images (four images from the first row, the first and the third images from the second row, the third image from the third row, and the first two images from the last row) are relevant to the concept. After the user clicks on the submit button in the learner frame, the fourth screen is displayed.
- Screen 4. Sampling and relevance feedback ends. Figure 5 shows that all returned images in the similarity search frame fit the query

concept (bird of paradise).

As observed, in two iterations, our system is able to retrieve fifteen relevant images from the image database. In this example, we use the keyword-based search engine to seed our perception-based search engine. The keyword-based search engine can be used to quickly identify the set of images relevant to the specified keywords. Based on these relevant images, the perception-based search engine can explore the feature space and discover more images relevant to the users' query concept.

The above example illustrates that our proposed system has three major features that are crucial for effective and efficient image retrieval:

(1) *Supporting flexible concept formulation.* One can search for a general flower concept or a specific kind of flower (e.g., bird of paradise). Also note in the Figure 4 that the system presents the flower of different angles and shapes on different backgrounds to ask for feedback. The system can intelligently explore more possibilities to refine the query concept.
(2) *Supporting different bootstrapping options.* Our system can support three bootstrapping options: query by keyword, query by example, and query by nothing. The example above shows that bootstrapping a perception-based search with a keyword search is helpful. However, our engine does not require a user to initiate a query with "good" seed images. Even if all initial randomly generated samples presented to the user by our system are negative, the negative-labeled instances can be used to shrink the pool of the next sampling round substantially. Therefore, the probability that a relevant example will be presented by our system in the next iteration will be considerably higher.
(3) *Accomplishing the above tasks quickly and accurately.* Our image-feature extractor and high-dimensional indexer (presented in Section 2) make both query-concept learning and image retrieval efficient and effective.

The rest of the chapter is organized as follows: In Section 2, we depict the components of our search engine. In Section 3, we present three learning algorithms, which power our perception-based image retrieval engine. Sections 4 and 5 report experimental results. Section 6 discusses related work. Finally, we offer our concluding remarks in Section 7.

2. System Architecture

Our proposed system offers three options for starting an image query, *query by nothing*, *query by keyword*, and *query by example*. Query by keyword and query by example may not always be practical, since good examples may be difficult to come by, and an image-query concept may be difficult to articulate in words. Our query-by-nothing paradigm uses an intelligent sampling process to find some relevant images, and is a necessary alternative when the other options are not available. Once some images relevant to the query concept are found, our perception-based search engine can formulate the query concept much more complete and in a user-friendly fashion as we showed in Section 1.1.

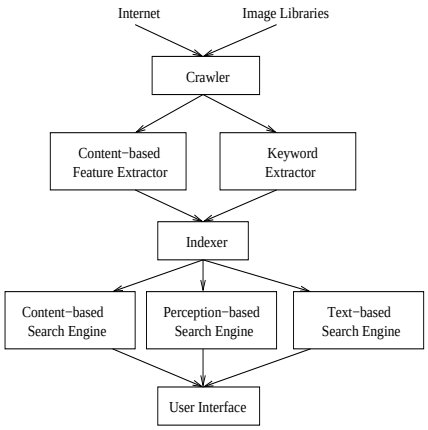


Fig. 6. System Components.

This section depicts the components of our system. Figure 6 presents the architecture, which consists of eight components: *crawler*, *keyword extractor*, *content-based feature extractor*, *indexer*, *perception-based search engine*, *content-based search engine*, *text-based search engine*, and *user interface*.

Not all image search engines employ all eight components above. As shown in Section 6, Google and Yahoo! presently support keyword-only image searches. Some systems such as VisualSEEk [36] can support keyword and content-based image retrieval. In this section, we briefly describe selected components, including the content-based feature extractor, high-dimensional indexer, perception-based search engine, and content-based

search engine. (We omit discussing components that are well understood such as crawler, keyword extractor, text-based search engine, and user interface.) In Sections 3, we will discuss in depth the active learning algorithm that powers the perception-based search engine.

2.1. *Feature Extractor*

Our feature extractor collects perceptual features from images. Common perceptual features include color, shape, texture, and the spatial layout of these features. Feature extraction can be performed off-line; however, since the number of images can be large, feature extraction should be both efficient and effective. In other words, we would like to extract features that can represent images well, and at the same time, keep the extraction time reasonably short.

We have two major types of features [8], color and texture. For color, we have 12 color bins for the 11 culture colors (*black*, *white*, *red*, *yellow*, *green*, *blue*, *brown*, *purple*, *pink*, *orange*, and *gray*) and one outlier bin. We characterize each color with color histograms, color mean (in HSV channels), color variance (in HSV channels), elongation and spreadness. Texture consists of vertical, horizontal and diagonal orientations. For each orientation, we have texture energy mean, variance, elongation and spreadness. This gives us 12 texture features. For each of these, we extract coarse, medium and fine texture features. Altogether, we have 108 color and 36 texture features [38].

2.2. *High-dimensional Indexer*

An indexer is essential to make both query-concept learning and image retrieval efficient. In addition, to take users' subjectivity into consideration, an indexer must support dynamic feature weighting. To deal with the "dimensionality-curse" problem and to support dynamic feature weighting, we propose an indexing scheme using clustering and classification methods for supporting *approximate similarity searches*. In many applications it is sufficient to perform an approximate search that returns many but not all nearest neighbors [1,11,10,21,26,27]. (A feature vector is often an approximate characterization of an object, so we are already dealing with approximations.) For instance, in content-based image retrieval [16,28,33,36] and document copy detection [4,9], it is usually acceptable to overlook a small fraction of the target objects. Thus it is not necessary to incur the high cost of an

exact search.

Our indexing method [19] is a statistical approach that works in two steps. It first performs non-supervised clustering using Tree-Structured Vector Quantization (TSVQ) [18] to group similar objects together. To maximize IO efficiency, each cluster is stored in a sequential file. A similarity search is then treated as a classification problem. Our hypothesis is that if a query object's class prediction yields C probable classes, then the probability is high that its nearest neighbors can be found in these C classes. This hypothesis is analogous to looking for books in a library. If we want to look for a calculus book and we know calculus belongs in the math category, by visiting the math section we can find many calculus books. Similarly, by searching for the most probable clusters into which the query object might be classified, we can harvest most of the similar objects.

2.3. *Perception-based Search Engine*

The perception-based search engine is the heart for supporting personalized image retrieval. The engine learns users' query concepts as learning a binary classifier that separates the images relevant to the query concept from the irrelevant ones. The learning takes place in an iterative process: The system presents examples to the users to refine the class boundary. The final class boundary is learned based on users' relevance feedback.

Relevance feedback is not new. Unfortunately, traditional relevance feedback methods require a large number of training instances to converge to a target concept. (Please refer to Section 6 for our discussion on related work.) In our perception-based engine, we explore several active learning algorithms, which can "grasp" a query profile with small number of training instances. We recently proposed two active learning algorithms, MEGA (The Maximizing Expected Generalization Algorithm) [7] and SVM_{Active} (Support Vector Machine Active Learning) [38] to tackle the problem effectively. In Section 3 we describe how MEGA and SVM_{Active} work. We further propose a novel technique, which combines the strengths of MEGA and SVM_{Active}.

2.4. *Content-based Search Engine*

The content-based search engine queries the image database using an image example selected by the user. Given a selected image, this engine finds all

similar images based on some appearance criteria such as color, texture, and shape. One limitation of the query-by-example approach is that it cannot capture semantic similarity. Again, this drawback can be overcome by the perception-based search engine. A query-by-example engine is still a useful tool if users just want to find images "appear" similar to a query image.

3. Active Learning Algorithms

We first examine two active learning algorithms: MEGA and SVM_{Active}. In Section 3.3, we will propose a hybrid approach that combines the strengths of MEGA and SVM_{Active}.

3.1. *MEGA*

The first challenge of query concept learning is to find some relevant objects so that the concept boundary can be fuzzily identified. Finding a relevant object can be difficult if only a small fraction of the dataset satisfies the target concept. We can improve the odds with an intelligent sampling method MEGA (The Maximizing Expected Generalization Algorithm), which finds relevant samples quickly, to initialize query-concept learning.

MEGA models query concepts in k-CNF [24], which can formulate virtually all practical query concepts[a]. MEGA uses k-DNF to bound the sampling space from which to select the most informative samples for soliciting user feedback.

Definition 1: k-**CNF**: For constant k, the representation class k-CNF consists of Boolean formulae of the form $c_1 \wedge \cdots \wedge c_\theta$, where each c_i is a disjunction of at most k literals over the Boolean variables $x_1, \ldots, x_n$. (A Boolean variable represents an image feature.) No prior bound is placed on θ.

Definition 2: k-**DNF**: For constant k, the representation class k-DNF consists of Boolean formulae of the form $d_1 \vee \cdots \vee d_\theta$, where each d_i is a conjunction of at most k literals over the Boolean variables $x_1, \ldots, x_n$. No prior bound is placed on θ.

MEGA initializes the query concept-space (QCS) as a k-CNF and the

[a] k-CNF is more expressive than k-term DNF, and it has both polynomial sample complexity and time complexity [25,31].

candidate concept-space (CCS) as a k-DNF. The QCS starts as the most specific concept and the CCS as the most general concept. The target concept is more general than the initial QCS and more specific than the initial CCS. The learner learns the QCS, while at the same time refining the CCS to delimit the boundary of the sampling space.

To learn a target concept, the well-known Valiant's algorithm [39] uses positive-labeled instances to refine QCS and negative-labeled instances to refine CCS. The key difference between Valiant's algorithm and MEGA lies in how training samples are selected. Valiant's algorithm employs a random sampling strategy. MEGA makes sure that each sample can be most useful, and hence reduces the number of samples needed to learn a concept by employing dual strategies.

1. Bounding the sample space: Avoid choosing useless unlabeled instances by using the CCS and QCS to delimit the sampling boundary.

2. Maximizing the usefulness of a sample: Choose an example that will remove the maximum expected number of disjunctive terms from QCS. In other words, we choose an example that can maximize the expected generalization of the concept. Even if the example is labeled negative by the user, it can be useful to remove conjunctive terms in the CCS.

It may appear that if we pick an example that has more dissimilar disjunctions (compared to the QCS), we would have a better chance of eliminating more disjunctive terms. This is, however, not true. An example must be labeled by the user as positive to be useful for refining QCS. Unfortunately, an example is less likely to be labeled positive when it has more disjunctions that are dissimilar to the target concept. Therefore, there is a tradeoff between choosing an example that has more contradictory terms and choosing one that is more likely to be labeled positive. The detailed MEGA algorithm is documented in [7].

3.2. SVM$_{Active}$

Once some relevant and some irrelevant samples are marked, we employ SVM$_{Active}$ to refine the class boundary. We add the active learning component to SVMs for selecting the most informative samples to query a user and to quickly refine a boundary that separates data objects that satisfy the user's query concept from the rest of the dataset.

For the purpose of query-concept learning, we consider SVMs in the bi-

nary classification setting. We are given training data $\{\mathbf{x}_1 \ldots \mathbf{x}_n\}$ that are vectors in some space $\mathcal{X} \subseteq \mathbb{R}^d$. We are also given their labels $\{y_1 \ldots y_n\}$ where $y_i \in \{-1, 1\}$. In their simplest form, SVMs are hyperplanes that separate the training data by a maximal margin. All vectors lying on one side of the hyperplane are labeled as -1, and all vectors lying on the other side are labeled as 1. The training instances that lie closest to the hyperplane are called *support vectors*. More generally, SVMs allow us to project the original training data in space $\mathcal{X}$ to a higher dimensional feature space $\mathcal{F}$ via a Mercer kernel operator K. In other words, we consider the set of classifiers of the form: $f(\mathbf{x}) = \sum_{i=1}^{n} \alpha_i K(\mathbf{x}_i, \mathbf{x})$. When $f(\mathbf{x}) \geq 0$ we classify $\mathbf{x}$ as $+1$, otherwise we classify $\mathbf{x}$ as -1.

When K satisfies Mercer's condition [5] we can write: $K(\mathbf{u}, \mathbf{v}) = \Phi(\mathbf{u}) \cdot \Phi(\mathbf{v})$ where $\Phi : \mathcal{X} \rightarrow \mathcal{F}$ and "$\cdot$" denotes an inner product. We can then rewrite f as:

$$f(\mathbf{x}) = \mathbf{w} \cdot \Phi(\mathbf{x}), \text{ where } \mathbf{w} = \sum_{i=1}^{n} \alpha_i \Phi(\mathbf{x}_i). \tag{1}$$

Thus, by using K we are implicitly projecting the training data into a different (often higher dimensional) feature space $\mathcal{F}$. The SVM then computes the α_is that correspond to the maximal margin hyperplane in $\mathcal{F}$. By choosing different kernel functions, we can implicitly project the training data from $\mathcal{X}$ into spaces $\mathcal{F}$ for which hyperplanes in $\mathcal{F}$ correspond to more complex decision boundaries in the original space $\mathcal{X}$. The detailed SVM_{Active} algorithm is documented in [38].

Intuitively, SVM_{Active} works by combining the following three ideas:

1. SVM_{Active} regards the task of learning a target concept as one of learning an SVM binary classifier. An SVM captures the query concept by separating the relevant images from the irrelevant images with a hyperplane in a projected space, usually a very high-dimensional one. The projected points on one side of the hyperplane are considered relevant to the query concept and the rest irrelevant.

2. SVM_{Active} learns the classifier quickly via active learning. The active part of SVM_{Active} selects the most informative instances with which to train the SVM classifier. This step ensures fast convergence to the query concept in a small number of feedback rounds.

3. Once the classifier is trained, SVM_{Active} returns the top-k most relevant images. These are the k images farthest from the hyperplane on the query

concept side.

3.3. *Hybrid Algorithms*

Although MEGA and SVM_{Active} are effective active learning algorithms, we believe combining their strengths can result in even better learning algorithms. In this section we propose a hybrid algorithm.

As discussed in Section 3.2, the SVM_{Active} scheme needs at least one positive and one negative example to start. MEGA is not restricted by this seeding requirement, and it is able to find relevant examples quickly by refining the sampling boundary. It is therefore logical to employ MEGA to perform the *initialization* task. Once some relevant images are found, the refinement step can be executed by either MEGA or SVM_{Active}.

Thus, we can have three execution alternatives.

(1) MEGA only. Use MEGA all the way to learn a concept.
(2) SVM_{Active} only. Use random sampling to find the first relevant example(s) and then use SVM_{Active} to learn a concept.
(3) Pipeline learning. Use MEGA to find initial relevant objects, and then switch to SVM_{Active} for refining the binary classifier and ranking returned objects.

4. Experiments

We implemented both MEGA and SVM_{Active} in C, $C++$, and tested using an Intel Pentium III^{TM} workstation running Linux. We have implemented an industrial strength prototype with all features discussed in this chapter.

We tested our prototype with the intent of answering three central questions.

(1) Do MEGA and SVM_{Active} learn to model complex query concepts in very high dimensional spaces both quickly and with a small number of iterations?
(2) Is using MEGA to find the first relevant object(s) and then switching to SVM_{Active} a more effective algorithm than using the two learning algorithms individually?

(3) Does the percentage of relevant data affect learning performance? (If we reduce the matching data from 5% of the dataset to 1%, does it take more iterations to learn query concepts?)

For our empirical evaluation of our learning methods we used a real-world image dataset: a twenty-category image dataset where each category consisted of 200 to 300 images. Images for this dataset were collected from Corel Image CDs and the Internet. The image set consists of *architecture*, *bears*, *cities*, *clouds*, *couples*, *flowers*, *insects*, *ladies*, *landscape*, *objectionable images*, *planets*, *tigers*, *tools*, *waves*, *elephants*, *fabrics*, *fireworks*, *food*, and *textures*.

We separate our queries into two categories: 5% and 1% queries. The matching images for each of the more specific query concepts such as "purple flowers" and "white bears" account for about 1% of the total dataset. More general concepts such as "bears", "flowers" and "architectures" have about 5% matching images in the dataset. For each experiment, we will report results for these two categories of queries separately.

In the experiment, we report the precision of the top-10 and top-20 retrievals to measure performance. To be objective, we run each experiment ten times and report the average precision.

5. Results and Discussion

Here we present our analysis of the results, organized with regard to the central questions listed at the start of Section 4.

5.1. MEGA, SVM$_{Active}$, *and Pipelining* MEGA *with* SVM$_{Active}$

For top-10 retrieval with 5% matching data (Figure 7(a)), SVM$_{Active}$ clearly outperforms MEGA. The major weakness of SVM$_{Active}$ is in initialization—finding the first few positive samples. Such weakness may not seriously affect this experiment, since there is a high probability of finding one of the 5% positive examples through random sampling. For queries with only 1% matching data (Figure 7(b)), such a weakness becomes more significant, because it substantially degrades SVM$_{Active}$'s performance. Especially, in the first two iterations, its precision is very low. Overall, the precision of MEGA and SVM$_{Active}$ is similar for 1% queries.

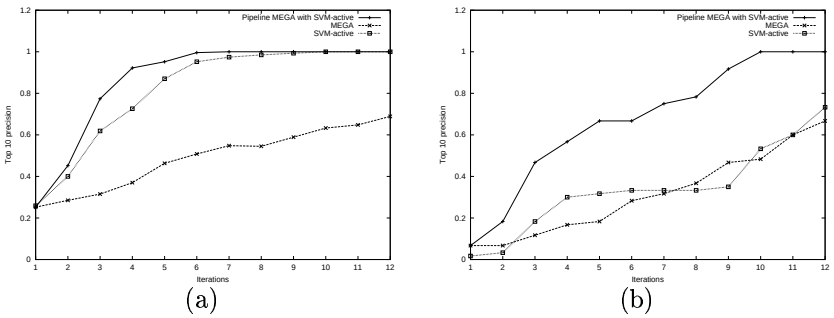


Fig. 7. Precision versus iterations for top-10 retrieval (a) 5% and (b) 1% queries.

The hybrid algorithm (pipelining MEGA with SVM_{Active}) clearly outperforms both SVM_{Active} and MEGA. The difference in precision is more significant for queries with 1% matching data than for those with 5% matching data. This trend indicates the strength of the hybrid algorithm in handling more specific query concepts and/or larger datasets. Note that the precision of the hybrid algorithm reaches 100% after 6 and 10 iterations, respectively, for these two experiments (Figures 7(a) and (b)).

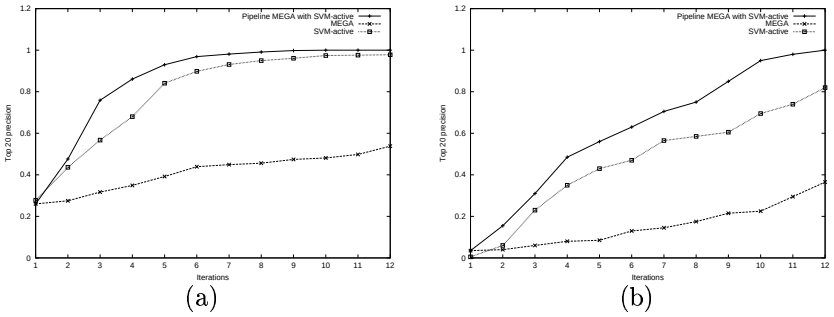


Fig. 8. Precision versus iterations for top-20 retrieval (a) 5% and (b) 1% queries.

For top-20 retrieval (Figures 8(a) and (b)), the precision of the hybrid algorithm remains the highest, followed by SVM_{Active} and MEGA. Again, the differences in their performances are more significant for queries with 1% matching data than for those with 5% matching data. For 1% queries, the hybrid algorithm achieves 100% precision after 12 iterations, which indicates the robustness of this new enhancement.

5.2. *Observations*

Our experiments have answered the four questions that we stated in the beginning of Section 4.

(1) MEGA and SVM_{Active} can learn complex query concepts in high-dimensional spaces in a small number of user iterations. Compared to some recent schemes proposed by the database community such as Mindreader [22] and Falcon [40], MEGA and SVM_{Active} perform substantially better.
(2) The hybrid scheme that uses MEGA to find the first relevant object(s) and then switches to SVM_{Active} works significantly better than using the two learning algorithms individually.
(3) When the matching data is scarce, the number of iterations required to learn a concept increases.

6. Related Work

Figure 9 depicts the traditional image search engines. A shaded box in the figure indicates that the component is used in the search paradigm. Figure 9(a) shows the architecture of the current google image-search engine. Google uses text only, and does not consider image content. For a text-based image retrieval engine, keywords entered by users may have multiple senses and hence may not precisely characterize the users' query concept. For example, the search results with the keyword "baby" return images ranging from baby photos to baby drawings, statues, and cartoons.

Figure 9(b) shows the architecture of a typical content-based image retrieval system. The following systems fit into this category: IBM QBIC [16], Stanford SIMPLIcity [40], Columbia VisualSEEk and WebSEEL [36], CMU Informedia [37] and UCSB NeTra [29]. Most systems assume that users can provide a "good" example to start a query. This requirement may not be realistic, since if good examples can be found easily, then the search engine is not needed. In addition, these systems assume that similar images have similar visual features. This restrictive assumption may not be able to model a query concept such as flowers that have diversified visual features.

Our system (Figure 6) employs intelligent sampling methods to minimize the number of samples required to learn a binary classifier that separates the objects matching the concept from the rest. To improve clas-

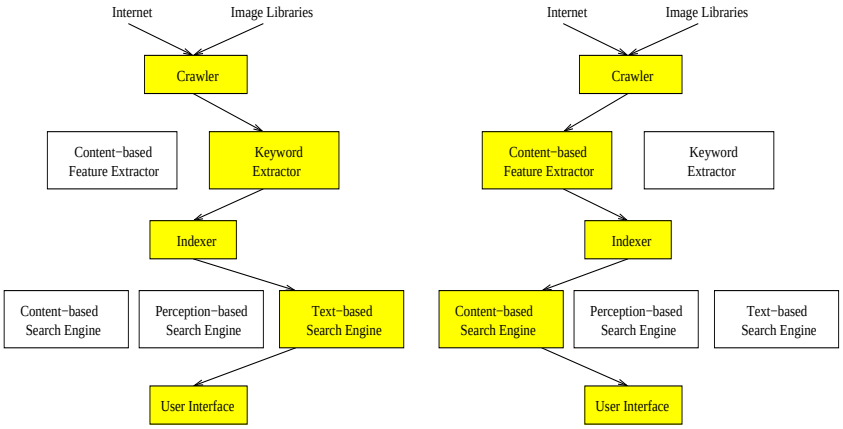


Fig. 9. System Architecture of Three Search Paradigms.

sification accuracy, researchers recently proposed a number of ensemble techniques such as *bagging* [2], *arcing* [3], and *boosting* [34,20]. These ensemble schemes enjoy success in improving classification accuracy through reduction in bias or variance, but they do not help reduce the number of samples required to learn a query concept. In fact, most ensemble schemes actually increase learning time because they introduce learning redundancy for improving prediction accuracy [14,20,32].

To reduce the number of required samples, researchers in the machine learning community have conducted several studies of active learning for classification. The *query by committee* algorithm [35,17] uses a distribution over all possible classifiers and attempts to greedily reduce the entropy of this distribution. This general purpose algorithm has been applied in a number of domains[b] using classifiers (such as Naive Bayes classifiers [13,30]) for which specifying and sampling classifiers from a distribution is natural. Probabilistic models such as the Naive Bayes classifier provide interpretable models and principled ways to incorporate prior knowledge and data with missing values. However, they typically do not perform as well as discriminative methods such as SVMs [23,15].

For image retrieval, the PicHunter system [12] uses Bayesian prediction to infer the goal image, based upon the users' actions. The system shows that employing active learning can drastically cut down the number of iterations

[b]Although, to our knowledge, not to the image retrieval domain.

(up to 80% in some experiments). However the authors also point out that their scheme is computationally intensive, since it needs to recompute the conditional probability for all unlabeled samples after each round of user feedback and hence may not scale well with larger dataset sizes. Our system reduces the sampling cost substantially by employing our high-dimensional indexing scheme (Section 2.2).

7. Conclusion

This chapter proposes an image retrieval system that uses active learning to capture complex and subjective query concepts. We have proposed using MEGA followed by SVM_{Active} as our perception-based search engine. Our system uses MEGA to quickly find objects relevant to the query concept, then switches to SVM_{Active} once relevant objects are found. Our experimental results show that this hybrid approach outperforms MEGA and SVM_{Active} when they are used separately. We have also presented key supporting components—a multi-resolution image-feature extractor and a high-dimensional indexer— for making both query-concept learning and image retrieval effieicnt. An on-line system prototype is available at [6].

Unlike traditional data mining tasks in which training data may be abundant, in an online training task, training data may be meager. We believe that being able to learn complex and subjective query concepts despite restrictions of time and paucity of samples, can make a search engine more user-friendly and personalized. Many applications can benefit from this technology. In e-commerce applications it could help customers quickly locating the desired products; in crime fighting it could help with identification; in medicine it could help distinguish abnormalities.

References

1. S. Arya, D. Mount, N. Netanyahu, R. Silverman, and A. Wu. An optimal algorithm for approximate nearest neighbor searching in fixed dimensions. *Proceedings of the 5th SODA*, pages 573–82, 1994.
2. L. Breiman. Bagging predicators. *Machine Learning*, pages 123–140, 1996.
3. L. Breiman. Arcing classifiers. *The Annals of Statistics*, pages 801–849, 1998.
4. S. Brin and H. Garcia-Molina. Copy detection mechanisms for digital documents. *Proceedings of ACM SIGMOD*, May 1995.
5. C. Burges. A tutorial on support vector machines for pattern recognition. *Data Mining and Knowledge Discovery*, 2:121–167, 1998.

6. E. Chang, K.-T. Cheng, W.-C. Lai, C.-T. Wu, C.-W. Chang, and Y.-L. Wu. PBIR — a system that learns subjective image query concepts. *Proceedings of ACM Multimedia, http://www.mmdb.ece.ucsb.edu/~demo/corelacm/*, pages 611–614, October 2001.
7. E. Chang and B. Li. Mega — the maximizing expected generalization algorithm for learning complex query concepts (extended version). *Technical Report http://www-db.stanford.edu/∼echang/mega.pdf*, November 2000.
8. E. Chang, B. Li, and C. Li. Towards perception-based image retrieval. *IEEE Content-Based Access of Image and Video Libraries*, pages 101–105, June 2000.
9. E. Chang, J. Wang, C. Li, and G. Wiederhold. RIME - a replicated image detector for the www. *Proc. of SPIE Symposium of Voice, Video, and Data Communications*, November 1998.
10. P. Ciaccia and M. Patella. Pac nearest neighbor queries: Approximate and controlled search in high-dimensional and metric spaces. *Proceedings of ICDE*, pages 244–255, 2000.
11. K. Clarkson. An algorithm for approximate closest-point queries. *Proceedings of the 10th SCG*, pages 160–64, 1994.
12. I. J. Cox, M. L. Miller, T. P. Minka, T. V. Papathomas, and P. N. Yianilos. The bayesian image retrieval system, pichunter: Theory, implementation and psychological experiments. *IEEE Transaction on Image Processing (to appear)*, 2000.
13. I. Dagan and S. Engelson. Committee-based sampling for training probabilistic classifiers. In *Proceedings of the Twelfth International Conference on Machine Learning*, pages 150–157. Morgan Kaufmann, 1995.
14. T. Dietterich and G. Bakiri. Solving multiclass learning problems via error-correcting output codes. *Journal of Artifical Intelligence Research*, 2, 1995.
15. S. Dumais, J. Platt, D. Heckerman, and M. Sahami. Inductive learning algorithms and representations for text categorization. In *Proceedings of the Seventh International Conference on Information and Knowledge Management*. ACM Press, 1998.
16. M. Flickner, H. Sawhney, J. Ashley, Q. Huang, B. Dom, M. Gorkani, J. Hafner, D. Lee, D. Petkovic, D. Steele, and P. Yanker. Query by image and video content: the QBIC system. *IEEE Computer*, 28(9):23–32, 1995.
17. Y. Freund, H. Seung, E. Shamir, and N. Tishby. Selective sampling using the Query by Committee algorithm. *Machine Learning*, 28:133–168, 1997.
18. A. Gersho and R. Gray. *Vector Quantization and Signal Compression*. Kluwer Academic, 1991.
19. K. Goh and E. Chang. Indexing multimedia data in high-dimensional and dynamic weighted feature spaces. *The* 6th *Visual Database Conference*, May 2002.
20. A. Grove and D. Schuurmans. Boosting in the limit: Maximizing the margin of learned ensembles. In *Proc. 15th National Conference on Artificial Intelligence (AAAI)*, 1998.
21. P. Indyk and R. Motwani. Approximate nearest neighbors: Towards removing the curse of dimensionality. *Proceedings of the 30th STOC*, pages 604–13,

1998.

22. Y. Ishikawa, R. Subramanya, and C. Faloutsos. Mindreader: Querying databases through multiple examples. *VLDB*, 1998.
23. T. Joachims. Text categorization with support vector machines. In *Proceedings of the European Conference on Machine Learning*. Springer-Verlag, 1998.
24. M. Kearns, M. Li, and L. Valiant. Learning boolean formulae. *Journal of ACM*, 41(6):1298–1328, 1994.
25. M. Kearns and U. Vazirani. *An Introduction to Computational Learning Theory*. MIT Press, 1994.
26. J. M. Kleinberg. Two algorithms for nearest-neighbor search in high dimensions. *Proceedings of the 29th STOC*, 1997.
27. E. Kushilevitz, R. Ostrovsky, and Y. Rabani. Efficient search for approximate nearest neighbor in high dimensional spaces. *Proceedings of the 30th STOC*, pages 614–23, 1998.
28. J. Li, J. Z. Wang, and G. Wiederhold. Irm: Integrated region matching for image retrieval. *Proceedings of ACM Multimedia*, October 2000.
29. W. Y. Ma and B. Manjunath. Netra: A toolbox for navigating large image database. *Proc. Int'l Conf. Image Processing*, pages 568–571, 1997.
30. A. McCallum and K. Nigam. Employing EM in pool-based active learning for text classification. In *Proceedings of the Fifteenth International Conference on Machine Learning*. Morgan Kaufmann, 1998.
31. T. Michell. *Machine Learning*. McGraw Hill, 1997.
32. M. Moreira and E. Mayoraz. Improving pairwise coupling classification with error correcting classifiers. *Proceedings of the Tenth European Conference on Machine Learning*, April 1998.
33. A. Natsev, R. Rastogi, and K. Shim. Walruc: A similarity retrieval algorithm for image databases. *Proceedings of ACM Sigmod*, June 1999.
34. R. Schapire, Y. Freund, P. Bartlett, and W. Lee. Boosting the margin: A new explanation for the effectiveness of voting methods. In *Proceeding of the Fourteenth International Conference on Machine Learning*. Morgan Kaufmann, 1997.
35. H. Seung, M. Opper, and H. Sompolinsky. Query by committee. In *Proceedings of the Fifth Workshop on Computational Learning Theory*, pages 287–294. Morgan Kaufmann, 1992.
36. J. R. Smith and S.-F. Chang. Visualseek: A fully automated content-based image query system. *ACM Multimedia Conference*, 1996.
37. S. Stevens, M. Christel, and H. Wactlar. Informedia: Improving access to digital video. *Interactions*, 1(3):67–71, 1994.
38. S. Tong and E. Chang. Support vector machine active learning for image retrieval. *Proceedings of ACM International Conference on Multimedia*, pages 107–118, October 2001.
39. L. Valiant. A theory of learnable. *Proceedings of the Sixteenth Annual ACM Symposium on Theory of Computing*, pages 436–445, 1984.
40. L. Wu, C. Faloutsos, K. Sycara, and T. R. Payne. Falcon: Feedback adaptive loop for content-based retrieval. *The* 26th *VLDB*, September 2000.